

# Data Quest

## Mastering Python Collections

*Summary: Journey through the digital realm as a data engineer! Master Python's powerful data structures while building and processing game data.*

*Version: 3.0*

# Contents


**I Foreword** **2**

**II AI Instructions** **3**

**III Introduction** **5**

**IV Common Instructions** **6**
IV.1 General Rules . . . . . 6
IV.2 Additional Guidelines . . . . . 6

**V Exercise 0: Command Quest** **7**

**VI Exercise 1: Score Cruncher** **9**

**VII Exercise 2: Position Tracker** **11**

**VIII Exercise 3: Achievement Hunter** **13**

**IX Exercise 4: Inventory Master** **15**

**X Exercise 5: Stream Wizard** **17**

**XI Exercise 6: Data Alchemist** **19**

**XII Turn in and Submission** **21**

# Chapter I

# Foreword

In Twitter's early growth days, scaling was not just about adding more servers. It was about fixing tiny decisions that suddenly mattered at massive scale. A pattern that showed up more than once was using simple sequential data structures (like lists or arrays) because they were easy and perfectly fine for small traffic. Then the user base exploded. Operations like "is this already present?" that once took microseconds started running millions of times per second, quietly turning into performance bottlenecks. Nothing was logically wrong: the code was clean and correct, but the wrong container turned linear time into a production headache. Swapping in hash-based structures often fixed the issue almost magically. It's a great reminder that at scale, your data structure is your algorithm.

# Chapter II

# AI Instructions

## Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

## Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

## Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

## Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

## Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

**✓ Good practice:**

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

**✗ Bad practice:**

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

**✓ Good practice:**

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

**✗ Bad practice:**

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

# Chapter III

# Introduction

Welcome back to the digital realm, data engineer!

Your journey through Python's foundations has prepared you well. You have mastered the basic syntax that powers digital gardens, built robust class hierarchies that model real-world systems, and learned to handle the unexpected with graceful exception management. Now, you're ready to tackle the heart of data engineering: **collections and data structures** that you will explore in a gaming environment.

Picture this: In 1980, Pac-Man's entire game state—every dot, ghost position, and score—fit in just 16KB of RAM. The programmers had to be wizards of efficiency! They discovered that organizing data was not just about saving memory: it was about unlocking gaming magic. Fast-forward to today: Fortnite processes over 10 million concurrent players, each generating thousands of data points per second. Same principles, bigger playground!

Python's collection types are designed for various use cases, and each has its own specific features: **lists** (ordered, indexed, expandable), **tuples** (ordered, immutable, hashable), **sets** (unordered collections of unique elements), and **dictionaries** (key-values pairs).

On top of these come **generators** and **comprehensions**, adding powerful syntax and behaviors.

In this quest, you'll build the components to support a game analytics platform. Every exercise unlocks a new data type, and by the end, you'll be wielding Python collections like a data engineer!



`Starting with this project, and when properly introduced by an exercise, you will be able to use each new Python data structure and all its associated class methods.`

# Chapter IV

# Common Instructions

## IV.1 General Rules

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- All functions and methods must include type hints; check this using `mypy` .
- Your functions should handle exceptions gracefully to avoid crashes.
- For this project, you will need access to command-line parameters. You will do this by using the **sys** module through the *import* mechanism. Imports will be addressed in more detail in a future project.
- No file I/O operations are allowed. All data must be processed in-memory or via command-line arguments.
- Focus on demonstrating collection usage patterns clearly.
- Show both basic operations and advanced techniques for each data structure.



`The following standard types are allowed, along with all their associated methods and constructors: str, int, float.`

## IV.2 Additional Guidelines

- Submit your work to the assigned Git repository.
- Only the content in this repository will be evaluated.

# Chapter V

# Exercise 0: Command Quest

|  | Exercise0 |
|---|---|
| ft_command_quest | |
| Directory: *ex*0/ | |
| Files to Submit: `ft_command_quest.py` | |
| Authorized: `import sys, sys.argv, len(), print()` | |

**Welcome, Data Adventurer!** Every epic quest begins with understanding your tools. In the digital realm, programs need to receive instructions from the outside world. Your first mission is to discover how programs can receive messages from their users!

It is now time to introduce *lists*. Before building your own lists, let's manipulate a list that already exists: the command-line parameters, available through the module `sys`. The structure is similar to the one in C: an array of strings. Explore how to access and manipulate list elements.

Build a simple script that shows the data received as command-line parameters. Mimic the example below.

```
$> python3 ft_command_quest.py
=== Command Quest ===
Program name: ft_command_quest.py
No arguments provided!
Total arguments: 1

$> python3 ft_command_quest.py hello world 42
=== Command Quest ===
Program name: ft_command_quest.py
Arguments received: 3
Argument 1: hello
Argument 2: world
Argument 3: 42
Total arguments: 4

$> python3 ft_command_quest.py "Data Quest"
=== Command Quest ===
Program name: ft_command_quest.py
Arguments received: 1
Argument 1: Data Quest
Total arguments: 2
```



Simply use "import sys" at the top of your script in order to access the sys.argv list.



There are multiple ways to avoid printing the program name again with the arguments. Prepare to discuss alternate solutions during the evaluation.

# Chapter VI

# Exercise 1: Score Cruncher

|  | Exercise1 |
|---|---|
| ft_score_analytics | |
| Directory: *ex*1/ | |
| Files to Submit: `ft_score_analytics.py` | |
| Authorized: `import sys, sys.argv, len(), sum(), max(), min(), print()` | |

**Mission Briefing**: Now that you have mastered command communication, it is time for a data cleanup! Indeed, the user sending commands to the program is human and can make mistakes.

This exercise requires you to use lists to store scores and try/except blocks to handle invalid input gracefully (e.g., when a user provides non-numeric values).

You will get game scores as command-line parameters. You need to:

- Process the command-line arguments
- Handle the various erroneous cases (no arguments, non-numeric values) with appropriate messages
- Create a new *list* to store and organize the scores
- Calculate some basic stats that would make any game player happy (number, total, average, max, min, and range)
- Make the output look cool enough to impress your gaming buddies (you can mimic the example again)
- If both valid and invalid inputs are provided via the command line, discard the invalid ones and proceed with the remaining valid inputs unless none remain.

```
$> python3 ft_score_analytics.py 1500 2300 1800 2100 1950
=== Player Score Analytics ===
Scores processed: [1500, 2300, 1800, 2100, 1950]
Total players: 5
Total score: 9650
Average score: 1930.0
High score: 2300
Low score: 1500
Score range: 800

$> python3 ft_score_analytics.py
=== Player Score Analytics ===
No scores provided. Usage: python3 ft_score_analytics.py <score1> <score2> ...

$> python3 ft_score_analytics.py ab ac
=== Player Score Analytics ===
Invalid parameter: 'ab'
Invalid parameter: 'ac'
No scores provided. Usage: python3 ft_score_analytics.py <score1> <score2> ...
```

# Chapter VII

# Exercise 2: Position Tracker

|  | Exercise2 |
|---|---|
| ft_coordinate_system | |
| Directory: *ex2/* | |
| Files to Submit: `ft_coordinate_system.py` | |
| Authorized: `import math, math.sqrt(), input(), round(), print()` | |

**Level Up!** Time to master 3D coordinates! Remember playing games where you teleport to specific locations in a 3D world? Or when you need to find the distance between two points in 3D space? That's exactly what we're building!

This exercise requires the use of *tuples* to store 3D coordinates (x, y, z).

First, write a function `get_player_pos()` that:

- Asks the user for the new player coordinates in the format `x,y,z`
- Handles improper inputs
- Retries until a valid set of coordinates is provided
- Returns a tuple containing the player's current 3D coordinates

Then your code will:

- Get a first set of coordinates
- Display the tuple then display each coordinate separately
- Calculate the distance to the 3D center (0, 0, 0) (see below)
- Get a new set of coordinates
- Calculate the distance between the second and the first sets of coordinates

**Distance Formula**: To calculate the distance between two 3D points, we use the Euclidean distance formula $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$ .

For points `(x1, y1, z1)` and `(x2, y2, z2)`, the distance is `math.sqrt((x2-x1)**2 + (y2-y1)**2 + (z2-z1)**2)`. This is just the 3D extension of the Pythagorean theorem!

```
$> python3 ft_coordinate_system.py
=== Game Coordinate System ===

Get a first set of coordinates
Enter new coordinates as floats in format 'x,y,z': hello world
Invalid syntax
Enter new coordinates as floats in format 'x,y,z': 1.0 , 2.5, 3.0
Got a first tuple: (1.0, 2.5, 3.0)
It includes: X=1.0, Y=2.5, Z=3.0
Distance to center: 4.0311

Get a second set of coordinates
Enter new coordinates as floats in format 'x,y,z': 4,abc,5
Error on parameter 'abc': could not convert string to float: 'abc'
Enter new coordinates as floats in format 'x,y,z': 4,5,6
Distance between the 2 sets of coordinates: 4.9244
```



Just use 'import math' at the top of your script in order to use `math.sqrt()`.



Tuples are like data written in stone. Once created, they won't change.

# Chapter VIII

# Exercise 3: Achievement Hunter

|  | Exercise3 |
|---|---|
| ft_achievement_tracker | |
| Directory: *ex*3/ | |
| Files to Submit: `ft_achievement_tracker.py` | |
| Authorized: `len()`, `print()`, `import random`, `random.*`, `set()`, `set.union()`, `set.intersection()`, `set.difference()` | |

**Achievement Unlocked!** Time to build the coolest achievement system ever! You know how satisfying it is when you unlock that rare achievement? Now you're building the system that tracks them all!

This exercise requires the use of *sets* to store unique achievements and perform operations (union, intersection, difference) to analyze achievement collections across players.

Create a function `gen_player_achievements()` that will use a large fixed list of achievements to randomly assign a set to a player. Choose a random number of achievements, then pick this number of achievements from the list to build and return the *set*.

Then your code will:

- Generate achievement sets for a minimum of four different players
- Track unique achievements among all the players
- Find achievements shared by all players
- For each player, spot the achievements no one else has
- For each player, list the missing achievements to have them all

```
$> python3 ft_achievement_tracker.py
=== Achievement Tracker System ===

Player Alice: {'Crafting Genius', 'World Savior', 'Master Explorer', 'Collector Supreme', 'Untouchable',
      'Boss Slayer'}
Player Bob: {'Crafting Genius', 'Strategist', 'World Savior', 'Master Explorer', 'Unstoppable', '
    Collector Supreme', 'Untouchable'}
Player Charlie: {'Strategist', 'Speed Runner', 'Survivor', 'Master Explorer', 'Treasure Hunter', 'First
    Steps', 'Collector Supreme', 'Untouchable', 'Sharp Mind'}
Player Dylan: {'Strategist', 'Speed Runner', 'Unstoppable', 'Untouchable', 'Boss Slayer'}

All distinct achievements: {'Crafting Genius', 'Strategist', 'World Savior', 'Speed Runner', 'Survivor',
      'Master Explorer', 'Treasure Hunter', 'Unstoppable', 'First Steps', 'Collector Supreme', '
    Untouchable', 'Sharp Mind', 'Boss Slayer'}

Common achievements: {'Untouchable'}

Only Alice has: set()
Only Bob has: set()
Only Charlie has: {'Survivor', 'Treasure Hunter', 'First Steps', 'Sharp Mind'}
Only Dylan has: set()

Alice is missing: {'Strategist', 'Speed Runner', 'Survivor', 'Treasure Hunter', 'Unstoppable', 'Hidden
    Path Finder', 'First Steps', 'Sharp Mind'}
Bob is missing: {'Speed Runner', 'Survivor', 'Treasure Hunter', 'Hidden Path Finder', 'First Steps', '
    Sharp Mind', 'Boss Slayer'}
Charlie is missing: {'Crafting Genius', 'World Savior', 'Hidden Path Finder', 'Unstoppable', 'Boss
    Slayer'}
Dylan is missing: {'Crafting Genius', 'World Savior', 'Survivor', 'Master Explorer', 'Treasure Hunter',
    'Hidden Path Finder', 'First Steps', 'Collector Supreme', 'Sharp Mind'}
```



Adjust the total number of achievements and how many you pick up for each player, so that all the requested sets are likely to be non-empty. By the way, how does Python print an empty set, and why?

# Chapter IX

# Exercise 4: Inventory Master

|  | Exercise4 |
|---|---|
| ft_inventory_system | |
| Directory: *ex*4/ | |
| Files to Submit: `ft_inventory_system.py` | |
| Authorized: `import sys, sys.argv, len(), print(), sum(), list(), round(), dict.keys(), dict.values(), dict.update()` | |

**Loot Time!** Remember organizing your inventory in RPGs? Checking if you have that legendary sword? Time to build the ultimate inventory system!

This exercise requires the use of *dictionaries* to store inventory data.

Your code will first parse the command-line parameters to fill your inventory system. Each parameter must follow this format: `<item_name>:<quantity>` . Discard invalid parameters (invalid syntax, redundant parameters) with an error message, and place valid ones in a dictionary. The `<quantity>` values in the dictionary will be stored as an `int` so that calculations can be performed later.

Time to operate on your inventory:

- Display your inventory
- Create and display the list of all items that are part of your inventory
- Calculate and print out the total quantity of all items in the inventory
- Display for each item the quantity percentage it represents in the inventory
- Report the most and least abundant items (choosing the first from the command line in case of a tie)
- Finally, add a new item to your inventory and display it again

```
$> python3 ft_inventory_system.py sword:1 potion:5 shield:2 armor:3 helmet:1 sword:2 hello key:value
=== Inventory System Analysis ===
Redundant item 'sword' - discarding
Error - invalid parameter 'hello'
Quantity error for 'key': invalid literal for int() with base 10: 'value'
Got inventory: {'sword': 1, 'potion': 5, 'shield': 2, 'armor': 3, 'helmet': 1}
Item list: ['sword', 'potion', 'shield', 'armor', 'helmet']
Total quantity of the 5 items: 12
Item sword represents 8.3%
Item potion represents 41.7%
Item shield represents 16.7%
Item armor represents 25.0%
Item helmet represents 8.3%
Item most abundant: potion with quantity 5
Item least abundant: sword with quantity 1
Updated inventory: {'sword': 1, 'potion': 5, 'shield': 2, 'armor': 3, 'helmet': 1, 'magic_item': 1}
```



At the beginning of the game, your inventory is usually empty ;)

# Chapter X

# Exercise 5: Stream Wizard

|  | Exercise5 |
|---|---|
| ft_data_stream | |
| Directory: *ex*5/ | |
| Files to Submit: `ft_data_stream.py` | |
| Authorized: `next(), range(), len(), print(), import typing, typing.Generator, import random, random.*` | |

**Magic Time!** Ever wondered how games handle millions of events without crashing? Welcome to the world of **generators**, Python's memory-saving superpower!

This exercise requires the use of *generators* with the `yield` keyword to create data streams on the fly. You must implement generator functions that produce values on-demand rather than storing everything in memory.

Create an endless generator function `gen_event()` that picks a random name from a list of players, and a random action from a list of actions. Each time `next()` is called on this generator, it returns a new event as a tuple (name, action).

From the main part of your script, loop a thousand times and display all 1000 events you get from your `gen_event()` .

Then create a list of ten tuples generated by `gen_event()` again.

Finally, create a new generator function `consume_event` that takes the previously created list, randomly pick one of its elements, remove it from the list, and yields it, until the list is empty. This generator must be used directly in the `for .. in ..` construct.

```
$> python3 ft_data_stream.py
=== Game Data Stream Processor ===
Event 0: Player bob did action run
Event 1: Player alice did action eat
Event 2: Player bob did action sleep
Event 3: Player bob did action grab
Event 4: Player dylan did action run
Event 5: Player bob did action move
Event 6: Player alice did action move
Event 7: Player dylan did action move
Event 8: Player alice did action climb
Event 9: Player bob did action sleep
Event 10: Player bob did action run
Event 11: Player bob did action swim
Event 12: Player dylan did action swim
Event 13: Player charlie did action sleep
Event 14: Player charlie did action sleep
```

[...]

```
Event 992: Player dylan did action eat
Event 993: Player alice did action sleep
Event 994: Player charlie did action move
Event 995: Player charlie did action climb
Event 996: Player bob did action release
Event 997: Player bob did action grab
Event 998: Player bob did action move
Event 999: Player alice did action move
Built list of 10 events: [('charlie', 'move'), ('dylan', 'grab'), ('alice', 'use'), ('alice', 'use'), ('
    charlie', 'swim'), ('bob', 'run'), ('charlie', 'move'), ('dylan', 'climb'), ('alice', 'use'), ('bob
    ', 'release')]
Got event from list: ('charlie', 'swim')
Remains in list: [('charlie', 'move'), ('dylan', 'grab'), ('alice', 'use'), ('alice', 'use'), ('bob', '
    run'), ('charlie', 'move'), ('dylan', 'climb'), ('alice', 'use'), ('bob', 'release')]
Got event from list: ('alice', 'use')
Remains in list: [('charlie', 'move'), ('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('charlie',
    'move'), ('dylan', 'climb'), ('alice', 'use'), ('bob', 'release')]
Got event from list: ('charlie', 'move')
Remains in list: [('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('charlie', 'move'), ('dylan', '
    climb'), ('alice', 'use'), ('bob', 'release')]
Got event from list: ('charlie', 'move')
Remains in list: [('dylan', 'grab'), ('alice', 'use'), ('bob', 'run'), ('dylan', 'climb'), ('alice', '
    use'), ('bob', 'release')]
Got event from list: ('alice', 'use')
Remains in list: [('dylan', 'grab'), ('bob', 'run'), ('dylan', 'climb'), ('alice', 'use'), ('bob', '
    release')]
Got event from list: ('bob', 'release')
Remains in list: [('dylan', 'grab'), ('bob', 'run'), ('dylan', 'climb'), ('alice', 'use')]
Got event from list: ('bob', 'run')
Remains in list: [('dylan', 'grab'), ('dylan', 'climb'), ('alice', 'use')]
Got event from list: ('dylan', 'climb')
Remains in list: [('dylan', 'grab'), ('alice', 'use')]
Got event from list: ('alice', 'use')
Remains in list: [('dylan', 'grab')]
Got event from list: ('dylan', 'grab')
Remains in list: []
```

# Chapter XI

# Exercise 6: Data Alchemist

|  | Exercise6 |
|---|---|
| ft_data_alchemist | |
| Directory: *ex*6/ | |
| Files to Submit: `ft_data_alchemist.py` | |
| Authorized: `import random, random.*, print(), len(), sum(), round()` | |

**Final Boss Time!** You have mastered all the data structures. Now it's time to discover them in an elegant condensed form! This is where you become a true Data Alchemist!

This exercise requires the use of list and dictionary *comprehensions* to transform and filter data efficiently. These are fundamental Python features for data processing.

Create a list of player names, where some are capitalized and others are not. Build two list comprehensions: the first one creates a new list with all names capitalized, the second one creates a new list with only the capitalized names from the initial list.

Now, let's create a dictionary from this full capitalized list of player names. Names will be the keys, and values will be randomly generated scores in a defined range. Of course, a comprehension will build this dictionary. Then a second dict will be created, with scores higher than the average, also using a comprehension.

```
$> python3 ft_data_alchemist.py
=== Game Data Alchemist ===

Initial list of players: ['Alice', 'bob', 'Charlie', 'dylan', 'Emma', 'Gregory', 'john', 'kevin', 'Liam
    ']
New list with all names capitalized: ['Alice', 'Bob', 'Charlie', 'Dylan', 'Emma', 'Gregory', 'John', '
    Kevin', 'Liam']
New list of capitalized names only: ['Alice', 'Charlie', 'Emma', 'Gregory', 'Liam']

Score dict: {'Alice': 263, 'Bob': 666, 'Charlie': 907, 'Dylan': 170, 'Emma': 568, 'Gregory': 446, 'John
    ': 90, 'Kevin': 527, 'Liam': 54}
Score average is 410.11
High scores: {'Bob': 666, 'Charlie': 907, 'Emma': 568, 'Gregory': 446, 'Kevin': 527}
```



It is also possible to use comprehensions on sets.



Each comprehension should be on a single line (unless it exceeds the line size).

# Chapter XII

# Turn in and Submission

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Don't hesitate to double-check the names of your files to ensure they are correct.



During evaluation, you may be asked to explain data structure choices, demonstrate collection operations, or extend your analytics systems with new functionality. Make sure you understand the principles behind each data structure.



You need to return only the files requested by the subject of this project. Focus on clean, well-documented code that clearly demonstrates mastery of Python's collection types and data processing techniques.